

PROGRAMACION MOVIL

PARCIAL 2

JULIO CESAR VALENCIA OSORIO

JUAN STEVAN POSADA MARÍN

DOCENTE: RAUL GAVIRIA

UNIVERSIDAD LIBRE

SECCIONAL PEREIRA



Taller 01 · Hola Compose: Primera Pantalla Declarativa

Descripción general de la aplicación

La aplicación es una pantalla de presentación personal con estética de terminal de programación. Está compuesta por tres Composables principales: TarjetaBienvenida, que muestra un saludo con tipografía monoespaciada y acento verde sobre fondo oscuro; PantallaPresentacion, que incluye foto de avatar circular, nombre, cargo, descripción, botón de contacto e íconos de redes sociales; y ChatScreen, una lista de burbujas de mensajes interactivas (ChatBubble) que se expanden o contraen al tocarse gracias a `animateContentSize`. La paleta visual usa `#0D1117` como fondo, `#3FB950` como acento verde y `#161B22` para el fondo de las tarjetas, logrando el estilo de terminal que caracteriza el taller. No hay navegación: es una pantalla única pensada para validar el entorno de desarrollo y el uso de Composables básicos.

Prompts lanzados a la IA

Prompt 1 — Tarjeta de perfil con estilo terminal

"Crea un Composable en Jetpack Compose para una tarjeta de perfil personal. Debe incluir una foto circular usando `painterResource`, nombre, cargo y descripción. Añade tres íconos de redes sociales con `Icon()`. Aplica un estilo visual de terminal con fondo oscuro (`#0D1117`), texto verde (`#3FB950`) y tipografía monoespaciada."

La IA generó la estructura base del Composable con `Column` centrada, aplicó `Color(0xFF3FB950)` para el acento verde, usó `FontFamily.Monospace` en los textos y propuso `Box` con `clip(CircleShape)` para la foto circular. También incluyó `Surface` con `RoundedCornerShape` para la tarjeta contenedora y el borde sutil `BorderStroke(0.5.dp)`.

Prompt 2 — Lista de mensajes con burbujas expandibles

"Añade una lista de mensajes estilo chat con burbujas que se expanden al tocarse para mostrar el texto completo. La burbuja debe tener forma redondeada asimétrica y respetar la paleta oscura. Usa `LazyColumn` para el listado."

La IA generó `ChatBubble` con `remember { mutableStateOf(false) }` para el estado expandido, `animateContentSize()` sobre la `Surface` para la transición suave, y `maxLines` condicional en el `Text`. También generó `SampleData` con mensajes de ejemplo y `ChatScreen` integrando `TarjetaBienvenida` y la lista.

Lo que generó la IA

La IA construyó la estructura visual completa: la paleta de colores con los valores hexadecimales (líneas 13 a 18 del Taller 01), la foto circular con `clip(CircleShape)`, la distribución de íconos en `Row` con `Arrangement.spacedBy`, el efecto de burbuja expandible con `animateContentSize`, y el borde sutil con `BorderStroke(0.5.dp, Color(0xFF30363D))`.

Errores de la IA y correcciones realizadas

- **Paleta de colores ajustada (líneas 13–18):** La IA propuso una paleta hacker básica con verde genérico. Se ajustó el fondo a #0D1117 (negro más profundo) y el acento a #3FB950 (verde más vibrante) para mejorar el contraste. El comentario en el código dice: "La IA sugirió inicialmente una paleta hacker básica. Yo la ajusté para que el fondo sea un negro más profundo (#0D1117) y el verde sea más vibrante (#3FB950) para mejorar el contraste." *[Taller01.kt L13–18]*
- **Paddings insuficientes en TarjetaBienvenida (línea 42):** La IA generó los paddings con 8.dp, haciendo que los elementos se vieran comprimidos. Se aumentaron a 16.dp tanto en el modifier externo como en la Column interna. El comentario dice: "La IA generó esto con paddings de 8.dp. Los aumenté a 16.dp para que no se viera tan comprimido y tenga 'respiración visual'." *[Taller01.kt L42]*
- **verticalArrangement faltante en PantallaPresentacion (línea 68):** La IA generó una Column simple sin especificar alineación vertical, dejando el contenido en la parte superior de la pantalla. Se añadió verticalArrangement = Arrangement.Center. El comentario indica: "La IA sugirió una Column simple. Añadí verticalArrangement = Arrangement.Center para que la tarjeta de perfil se mantenga centrada y profesional en cualquier pantalla." *[Taller01.kt L68]*
- **Recurso de imagen inexistente (línea 76):** El código generado referenciaba R.drawable.profile, que no existía en el proyecto. Se corrigió a R.drawable.profile_picture, el nombre real del archivo en res/drawable. El comentario dice: "La IA intentó cargar R.drawable.profile (inexistente). Corregido al recurso real: R.drawable.profile_picture." *[Taller01.kt L76]*
- **Spacers insuficientes (línea 88):** Los Spacers originales eran de 8.dp entre elementos. Se aumentaron a 16.dp para mejorar el balance visual. El comentario en el código señala: "Se aumentó el tamaño de los Spacers de 8.dp a 16.dp para mejorar el balance visual." *[Taller01.kt L88]*
- **Ícono obsoleto Icons.Default.Twitter (línea 119):** La IA intentó usar Icons.Default.Twitter, que ya no está disponible en la librería estándar de Material Icons. Se reemplazó por Icons.Default.Share, Icons.Default.Person e Icons.Default.Info. El comentario dice: "La IA intentó usar Icons.Default.Twitter (no disponible). Se reemplazó por Share e Info de la librería estándar para evitar errores de compilación." *[Taller01.kt L119]*
- **Código generado en MainActivity (línea 148):** Toda la lógica fue generada inicialmente en MainActivity.kt sin separación por paquetes. Se realizó una refactorización manual para extraerlo al paquete taller01. El comentario en ChatScreen dice: "La IA generó este código inicialmente en MainActivity.kt. Realicé una refactorización manual para extraerlo a este paquete taller01." *[Taller01.kt L148]*

Taller 02 · Estado y Navegación: App de Lista de Tareas

Descripción general de la aplicación

La aplicación es un gestor de tareas personal con interfaz en modo oscuro Deep Dark. La pantalla principal (ListaTareasScreen) muestra las tareas en una LazyColumn con tarjetas, un estado vacío cuando no hay tareas y un FloatingActionButton azul para agregar. Cada tarjeta (TareaItem) combina un Checkbox con tachado de texto al completar, un botón de eliminar directo y soporte para swipe-to-delete mediante SwipeToDismissBox con fondo rojo animado. Al tocar una tarjeta se navega a DetalleTareaScreen, que muestra el título, el ID y un chip de estado. La navegación usa NavController con dos rutas: lista y detalle/{tareaId}. El estado persiste entre rotaciones gracias a rememberSaveable. La paleta usa fondo #0F111A, tarjetas #1E212E, acento azul #3D5AFE para acciones primarias y verde #4CAF50 para completadas.

Prompts lanzados a la IA

Prompt 1 — App de gestión de tareas en modo oscuro

"Crea una app de gestión de tareas con una interfaz moderna y limpia en modo oscuro. La lista debe verse profesional, con tarjetas para cada tarea, un botón flotante para añadir y navegación a una pantalla de detalles. Los títulos deben ser claros y en lenguaje natural."

La IA generó el esqueleto con Scaffold, TopAppBar, LazyColumn, FloatingActionButton y la data class Tarea. También generó la función Taller02App con NavHost como punto de entrada de la navegación con dos destinos.

Prompt 2 — TareaItem con swipe-to-delete y animación

"Crea un Composable TareaItem en Jetpack Compose con Checkbox a la izquierda, texto tachado si está completada, ícono de eliminar a la derecha y animación al marcar. Añade soporte para swipe-to-delete con SwipeToDismissBox y fondo rojo animado con animateColorAsState."

La IA generó TareaItem con SwipeToDismissBox, el fondo animado con animateColorAsState, el Checkbox con colores personalizados, TextDecoration.LineThrough condicional y el IconButton de eliminar. La estructura general fue correcta aunque requirió ajustes.

Prompt 3 — Corrección de excepción en navegación

"Al navegar de la lista al detalle, la app lanza IllegalArgumentException: 'navigation destination detalle/3 is unknown'. ¿Cuál es la causa?"

La IA identificó que el problema era una inconsistencia entre el string de ruta definido en NavHost (detalle/{tareaId}) y el string usado en navigate(). Se corrigió unificando el patrón usando interpolación directa: navController.navigate("detalle/\${tarea.id}").

Lo que generó la IA

La IA construyó el esqueleto completo de la aplicación: la estructura NavHost con dos destinos, la pantalla de lista con Scaffold y estado reactivo, el AlertDialog de nueva tarea con validación de campo vacío, el TareaItem con SwipeToDismissBox y fondo animado, y la pantalla de detalle. También sugirió `key = { it.id }` en la LazyColumn para renderizado eficiente y el mensaje de lista vacía centrado.

Errores de la IA y correcciones realizadas

- **Paleta de colores suavizada (líneas 10–17):** La IA generó una paleta heredada del taller anterior con verde brillante sobre fondo casi negro, causando contraste excesivo y fatiga visual. Se rediseñó a tonos azul-negro con acentos más suaves. El comentario dice: "Se suavizaron los tonos originales (verdes brillantes) a una gama azulada para mejorar la accesibilidad y reducir la fatiga visual tras detectar un contraste excesivo." *[Taller02.kt L10–17]*
- **Serializable en la data class Tarea (línea 22):** La IA no incluyó la implementación de Serializable en la data class Tarea, lo que impedía que rememberSaveable funcionara correctamente. Se añadió : Serializable. El comentario indica: "El modelo implementa Serializable para asegurar que el estado se mantenga correctamente durante los ciclos de vida de Android." *[Taller02.kt L22]*
- **remember reemplazado por rememberSaveable (línea 35):** La IA usó remember simple para la lista de tareas. Al rotar el dispositivo, toda la lista desaparecía. Se corrigió a rememberSaveable. El comentario en el código dice: "Se corrigió el uso de 'remember' simple por 'rememberSaveable' para evitar la pérdida de la lista al rotar el dispositivo." *[Taller02.kt L35]*
- **NavType.IntType explícito en la ruta (línea 44):** La IA definió el argumento de navegación sin especificar el tipo, lo que causaba errores de casting al recuperar el ID. Se añadió `type = NavType.IntType`. El comentario dice: "Se definió explícitamente NavType.IntType para evitar errores de casting al recuperar el ID en la pantalla de destino." *[Taller02.kt L44]*
- **Lenguaje de terminal en la UI (línea 71):** El código inicial usaba textos tipo `TASK_MANAGER >_` en el título del TopAppBar. Se reemplazó por lenguaje natural. El comentario dice: "Se cambió el lenguaje de la UI de 'TASK_MANAGER' a 'Gestor de Tareas' para que la interfaz sea más intuitiva y cercana al usuario." *[Taller02.kt L71]*
- **Ruta de navegación inconsistente (línea 112):** La IA generó la llamada a navigate con un string diferente al definido en el NavHost, provocando la excepción destination unknown. Se unificó el string de ruta. El comentario dice: "Se corrigió una excepción de 'destination unknown' unificando los strings de ruta en la llamada de navegación." *[Taller02.kt L112]*
- **Validación visual del campo de texto (línea 135):** El diálogo generado no mostraba ningún mensaje cuando el usuario intentaba guardar una tarea con el campo vacío. Se

añadió el Text de error con color AccentRed. El comentario dice: "Se añadió este mensaje de error para mejorar la validación visual de la entrada." *[Taller02.kt L135]*

- **Alcance de la animación en TareaItem (línea 188):** La animación de fade inicial actuaba sobre toda la Surface de la tarjeta, incluido el Checkbox. Se ajustó para que solo afecte al texto, manteniendo el Checkbox siempre visible. El comentario dice: "Se ajustó el alcance de la animación de fade: ahora solo afecta al texto para que el Checkbox se mantenga siempre visible con opacidad completa." *[Taller02.kt L188]*
- **Botón de eliminar directo añadido (línea 197):** La IA solo implementó swipe-to-delete, sin un botón visible de eliminar para usuarios que no conocen el gesto. Se añadió el IconButton directo complementario. El comentario dice: "Se añadió un botón de eliminar directo para mejorar la accesibilidad complementando el gesto de swipe." *[Taller02.kt L197]*
- **Ícono de navegación deprecado (línea 214):** La IA usó Icons.Default.ArrowBack, marcado como deprecated en versiones recientes de Material Icons. Se reemplazó por Icons.AutoMirrored.Filled.ArrowBack, que maneja automáticamente la dirección en dispositivos RTL. *[Taller02.kt L214]*

Taller 03 · ViewModel + StateFlow: App del Clima con API REST

Descripción general de la aplicación

La aplicación consume la API de OpenWeatherMap para mostrar datos meteorológicos en tiempo real. La pantalla (WeatherScreen) tiene un OutlinedTextField para buscar ciudades, con soporte de teclado ImeAction.Search. La lógica de red está en WeatherRepository, que instancia Retrofit directamente con la base URL de OpenWeatherMap y delega en WeatherApiService, una interfaz con un único método suspendido getWeather() que recibe ciudad, API key, unidades (metric) e idioma (es). Los datos se deserializan a WeatherResponse con las data classes Main, WeatherDesc y Wind, usando @SerializedName para mapear los campos del JSON. El estado se gestiona con WeatherViewModel, que expone un StateFlow de tipo sellado WeatherUiState con cuatro subtipos: Idle, Loading, Success y Error. El Composable lo observa con collectAsStateWithLifecycle() y renderiza según el subtipo activo. La pantalla de resultados (WeatherContent) muestra la ciudad, temperatura en grande, descripción y dos chips (WeatherChip) para humedad y viento. El fondo aplica un gradiente vertical cuyo color base cambia con animateColorAsState según la condición climática.

Prompts lanzados a la IA

Prompt 1 — Diseño visual de la pantalla del clima con gradiente animado

"Diseña una pantalla de clima en Jetpack Compose Material3 con: temperatura grande centrada, chips para humedad y viento, y un fondo con gradiente que cambia de color según la condición climática (soleado = naranja cálido, lluvia = azul, noche = morado oscuro). Usa animateColorAsState para la transición suave."

La IA generó la estructura de WeatherScreen con el Box de fondo gradiente y animateColorAsState con animationSpec = tween(1000). También generó WeatherContent con la temperatura en 72sp, los chips como Surface con fondo semitransparente y el bloque when para gestionar los estados de UI.

Prompt 2 — Sealed class WeatherUiState y consumo en el Composable

"Define un sealed class WeatherUiState con cuatro subtipos: Idle (object), Loading (object), Success con campo data de tipo WeatherResponse, y Error con campo message String. Muéstrame cómo consumirlo en un Composable con when y collectAsStateWithLifecycle."

La IA generó exactamente los cuatro subtipos incluyendo el Idle que no estaba en el enunciado original del PDF. También generó el bloque when en WeatherScreen con todos los casos cubiertos, usando val state = uiState para el smart cast y accediendo a state.data y state.message sin cast explícito.

Prompt 3 — Interfaz Retrofit y data classes para OpenWeatherMap

"Genera la interfaz Retrofit WeatherApiService para consumir <https://api.openweathermap.org/data/2.5/weather> con parámetros q,appid, units y lang. El

método debe ser suspendido. Crea también las data classes WeatherResponse, Main, WeatherDesc y Wind con @SerializedName mapeando los campos del JSON de OpenWeatherMap."

La IA generó WeatherApiService con @GET("data/2.5/weather") y los cuatro @Query correctos, y las data classes con los @SerializedName adecuados. También generó WeatherRepository instanciando Retrofit con GsonConverterFactory y la baseUrl correcta.

Lo que generó la IA

La IA construyó la capa de datos completa: WeatherApiService con los parámetros de query, WeatherRepository con la instancia de Retrofit y GsonConverterFactory, todas las data classes con sus @SerializedName, la sealed class WeatherUiState con los cuatro subtipos, el WeatherViewModel con viewModelScope.launch y manejo de excepciones, y la UI con el fondo gradiente animado, los chips y el bloque when de estados.

Errores de la IA y correcciones realizadas

- **Gradiente aplicado en Surface en lugar de Box contenedor (WeatherScreen.kt línea 47):** La IA colocó el Brush.verticalGradient directamente en el Modifier de la Surface de resultados, haciendo que el gradiente se cortara con los bordes redondeados y no cubriera toda la pantalla. Se movió al Box exterior principal. El comentario dice: "La IA aplicó el gradiente directamente en el Surface de los resultados. Esto causaba cortes visuales. Se movió el gradiente al Box contenedor principal para que cubra toda la pantalla." [*WeatherScreen.kt L47*]
- **Colores del gradiente refinados (WeatherScreen.kt líneas 28–37):** La IA propuso colores básicos para las condiciones climáticas. Se refinaron: el naranja fue ajustado a #E65100 (más cálido) y el azul de lluvia a #1A237E (más profundo y elegante). El comentario dice: "La IA sugirió colores básicos. Yo los refiné para que el naranja sea más cálido y el azul de lluvia sea un tono más profundo y elegante." [*WeatherScreen.kt L28–37*]
- **Temperatura en Kelvin por falta de units=metric (WeatherApiService.kt línea 8):** En la versión inicial la IA generó el método sin el parámetro @Query("units"), lo que hacía que la API retornara la temperatura en Kelvin. Se añadió @Query("units") units: String = "metric" y también @Query("lang") lang: String = "es" para recibir las descripciones en español. [*WeatherApiService.kt L8*]
- **WeatherUiState sin subtipo Idle (WeatherUiState.kt línea 4):** El sealed class generado inicialmente tenía solo Loading, Success y Error, sin un estado inicial. La pantalla no sabía qué mostrar antes de la primera búsqueda. Se añadió el subtipo Idle y se manejó en el bloque when de WeatherScreen con el mensaje informativo. [*WeatherUiState.kt L4*]
- **Sin validación de ciudad vacía en ViewModel (WeatherViewModel.kt línea 16):** La IA no incluyó ninguna validación antes de lanzar la coroutine. Con ciudad vacía, la API retornaba un error 400 poco informativo. Se añadió if (city.isBlank()) return al

inicio de loadWeather() para cortar la ejecución de forma temprana.
[WeatherViewModel.kt L16]

Taller 04 · Room + Hilt + Clean Architecture: App de Finanzas Personales

Descripción general de la aplicación

Esta aplicación es un sistema de registro de finanzas personales que permite al usuario registrar transacciones de tipo ingreso o gasto, categorizarlas (alimentación, transporte, ocio, etc.), y visualizar un resumen mensual en el dashboard principal. La pantalla de inicio muestra el balance actual, el total de ingresos y gastos del mes en tarjetas de resumen, y una gráfica de barras que compara ingresos vs gastos por semana. Existe una pantalla de listado de transacciones con filtros por categoría y tipo, y un formulario para agregar nuevas transacciones con validación de campos. Toda la información se persiste en una base de datos local con Room. La inyección de dependencias se gestiona con Hilt, y el proyecto está organizado en capas de Clean Architecture: data, domain y presentation, con paquetes claramente diferenciados.

Prompts lanzados a la IA

Prompt 1 — Sistema de diseño Material3

"Soy un estudiante de Android. Crea un sistema de diseño Material3 en Jetpack Compose para una app de finanzas personales: paleta de colores para tema claro y oscuro, definición de tipografía, shapes redondeados, y tres componentes reutilizables: TarjetaResumen (muestra monto y etiqueta), ChipCategoría (filtra por tipo) y una descripción de cómo integrar una gráfica de barras con la librería Vico."

La IA generó el archivo Theme.kt completo con ColorScheme para ambos temas, definición de Typography con fuentes Inter importadas desde Google Fonts, y los shapes. Los tres componentes fueron generados como Composables independientes y reutilizables. La descripción de integración con Vico incluía las dependencias necesarias en build.gradle y el uso básico de ColumnChart.

Prompt 2 — Definición de Entity y DAO con Room

"Crea la entidad Room TransaccionEntity con los campos: id autoincremental, descripcion String, monto Double, tipo String (INGRESO o GASTO), categoria String y fecha Long. Luego crea el DAO con métodos para insertar, eliminar, obtener todas las transacciones ordenadas por fecha descendente como Flow, y obtener la suma de montos filtrada por tipo como suspend function."

La IA generó exactamente lo solicitado con las anotaciones @Entity, @PrimaryKey, @Dao, @Query, @Insert y @Delete correctas. También sugirió añadir @ColumnInfo para los nombres de columnas en la base de datos.

Prompt 3 — Error de inyección Hilt

"Al compilar con Hilt me aparece el error: 'error: [Dagger/MissingBinding] TransaccionRepository cannot be provided without an @Inject constructor or an @Provides-annotated method'. ¿Por qué ocurre y cómo lo soluciono?"

La IA explicó que el error ocurre cuando el repositorio tiene una interfaz pero la implementación concreta no está vinculada en un módulo Hilt con @Binds o @Provides. Generó el módulo DataModule con @Binds para la interfaz y su implementación, resolviendo el error.

Lo que hizo la IA

La IA construyó el sistema de diseño completo con el Theme.kt, generó la estructura de base de datos con Room incluyendo entity, DAO y database, creó los Use Cases de dominio con validación de campos vacíos y montos negativos, y explicó el flujo de inyección de Hilt desde el módulo hasta el ViewModel. También generó el formulario de nueva transacción con validación reactiva de campos.

Errores de la IA y correcciones realizadas

- **API de Material2 en lugar de M3:** El Theme.kt generado usaba la función deprecated MaterialTheme.colors de Material2 en lugar de MaterialTheme.colorScheme de Material3. Se actualizaron todas las referencias a la API correcta de M3.
- **ViewModel sin anotaciones Hilt:** La IA generó el ViewModel sin la anotación @HiltViewModel ni el constructor anotado con @Inject, haciendo que Hilt no pudiera inyectarlo. Se añadieron las anotaciones correspondientes.
- **Dependencias de Android en la capa de dominio:** Los Use Cases generados tenían dependencias directas de Android (Context) en la capa de dominio, violando el principio de Clean Architecture. Se refactorizó para que el dominio dependiera únicamente de interfaces Kotlin puras.
- **Versión incompatible de Vico:** La gráfica de Vico requería una versión específica de la dependencia que no era compatible con la versión de Compose del proyecto. Se ajustó la versión en build.gradle tras consultar la documentación oficial.
- **Error 5 — libs.versions.toml incompleto** El archivo libs.versions.toml se cortó en la última línea del bloque [plugins], dejando la entrada del plugin ksp incompleta (ksp = { sin cerrar). Esto impedía que Gradle leyera el catálogo de versiones completo, generando el error Unresolved reference 'room' aunque las dependencias estaban correctamente declaradas. Solución: completar la línea con ksp = { id = "com.google.devtools.ksp", version = "2.0.21-1.0.28" } y sincronizar Gradle.
- **Error 6 — FlowRow no disponible** El componente FlowRow de Jetpack Compose requiere importación explícita de androidx.compose.foundation.layout.FlowRow y la anotación @OptIn(ExperimentalLayoutApi::class). Al no tenerlos, el compilador marcaba error de referencia no resuelta. Solución: reemplazar FlowRow por un

Column con chunked(3) que divide las categorías en filas de 3, eliminando la dependencia de APIs experimentales.

- **Error 7 — API de Vico 2.x incompatible** La librería de gráficos Vico versión 2.x cambió completamente su API respecto a versiones anteriores. Los métodos `CartesianChartModelProducer.build {}`, `rememberStartAxis()` y `rememberBottomAxis()` ya no existen. Solución: usar `CartesianChartModelProducer()` con `LaunchedEffect` y `runTransaction {}`, y reemplazar los ejes por `VerticalAxis.rememberStart()` y `HorizontalAxis.rememberBottom()`.
- **Error 8 — Paquete incorrecto en AndroidManifest** El Manifest tenía `android:name=".FinanzasApp"` pero las clases Kotlin usaban el paquete `com.unilibre.finanzasapp` en lugar de `com.finanzas.app`. Android Studio no encontraba las clases y marcaba error de referencia no resuelta en el Manifest. Solución: actualizar el namespace y `applicationId` en `build.gradle.kts` para que coincidieran con el paquete real del proyecto, o referenciar las clases con el paquete completo en el Manifest.
- **Error 9 — Crash al abrir la app en el dispositivo** La app crasheaba inmediatamente al abrirse. La causa era que el `AndroidManifest.xml` no tenía la anotación `android:name=".FinanzasApp"` en la etiqueta `<application>`, lo que impedía que Hilt se inicializara correctamente al arrancar la aplicación. Solución: agregar `android:name=".FinanzasApp"` en el Manifest.

Taller 05 · App Completa con IA Integrada: Asistente de Recetas Inteligente

Descripción general de la aplicación

Esta es la aplicación más compleja del módulo y actúa como proyecto integrador. El usuario abre la app y ve una cuadrícula de recetas guardadas como favoritas. Al presionar el botón flotante de cámara, accede a una pantalla de captura en tiempo real con CameraX. Toma una foto de los ingredientes que tiene disponibles y ML Kit analiza la imagen detectando los elementos con un nivel de confianza superior al 75%. La pantalla siguiente muestra la lista editable de ingredientes reconocidos, donde el usuario puede añadir o quitar items antes de confirmar. Al confirmar, la app construye un prompt y realiza una llamada a la API de Groq solicitando tres recetas posibles en formato JSON. La respuesta se muestra con un efecto typewriter animado, como si la IA estuviera escribiendo en tiempo real. Cada receta tiene su pantalla de detalle con los pasos, tiempo estimado, dificultad y opción de guardar como favorita en Room. La app aplica MVVM, Clean Architecture, Hilt y Room de forma integrada.

Prompts lanzados a la IA

Prompt 1 — Prototipado de pantallas (Uizard / descripción)

"Describe en detalle cómo debería verse visualmente cada una de las 5 pantallas de un asistente de recetas con IA: inicio con grid de favoritas, cámara con preview en tiempo real, lista de ingredientes detectados, pantalla de respuesta del LLM con efecto typewriter, y detalle de receta con pasos y botón de favorito. Usa lenguaje de diseño Material3."

La IA produjo una descripción detallada de cada pantalla con propuestas de layout, colores, tipografía y componentes Material3. Esta descripción sirvió como guía de referencia de diseño equivalente a un wireframe textual, y fue usada como input para Uizard para generar los mockups visuales.

Prompt 2 — Pantalla de streaming con efecto typewriter

"Implementa en Jetpack Compose la pantalla de resultado de recetas con efecto typewriter: el texto de la receta aparece carácter a carácter usando LaunchedEffect y un índice de estado. El fondo debe tener un blur sutil, y mostrar chips con el tiempo de preparación y nivel de dificultad. Incluye un botón animado de guardar que cambia de ícono al presionarse."

La IA generó el Composable RecetasIAScreen con el efecto typewriter implementado mediante LaunchedEffect que incrementa un índice sobre el texto y usa substring para mostrar solo los caracteres hasta ese punto. El blur de fondo fue implementado con Modifier.blur() disponible desde API 31.

Prompt 3 — Construcción del prompt para Gemini

"Crea la función Kotlin que construye el prompt para la API de Gemini dado una lista de ingredientes. El prompt debe pedir 3 recetas en formato JSON estricto con los campos: nombre, tiempo_minutos, dificultad (facil/media/dificil), pasos (lista de strings) y calorías_estimadas. Debe incluir la instrucción de no incluir texto adicional fuera del JSON."

La IA generó la función buildRecipePrompt que concatena los ingredientes y añade las instrucciones de formato JSON estricto, incluyendo la frase 'Responde únicamente con el JSON válido, sin bloques de código ni texto adicional'.

Prompt 4 — Error de permisos en tiempo de ejecución

"CameraX me lanza una SecurityException al intentar iniciar la vista previa: 'Permission denial: Camera'. Ya tengo el permiso declarado en el Manifest. ¿Cuál es el problema?"

La IA explicó que declarar el permiso en el Manifest no es suficiente en Android 6.0 y superior; es necesario solicitarlo en runtime con ActivityResultContracts.RequestPermission(). Generó el bloque de código completo con launcher y verificación del estado del permiso antes de iniciar CameraX.

Lo que hizo la IA

La IA fue de gran ayuda en este taller para generar el andamiaje inicial de CameraX con PreviewView y el binding del lifecycle, la integración de ML Kit Image Labeling con el procesamiento asíncrono de ImageProxy, la construcción del prompt estructurado para la

API de IA, el efecto typewriter animado, y la explicación de cómo parsear la respuesta JSON de Gemini con manejo de errores de deserialización. También describió visualmente las pantallas para guiar el prototipado en Uizard.

Errores de la IA y correcciones realizadas

- **API deprecated de CameraX:** El código de CameraX generado usaba la API deprecated de CameraX.bindToLifecycle sin el ProcessCameraProvider. Se actualizó al patrón correcto con ProcessCameraProvider.getInstance(context) y el listener de ListenableFuture.
- **Sin filtro de confianza en ML Kit:** ML Kit retornaba todos los labels sin filtro de confianza. La IA olvidó incluir el `.filter { it.confidence > 0.75f }` que se había especificado en el prompt. Se añadió el filtro manualmente para evitar ingredientes con reconocimientos poco confiables.
- **JSON envuelto en markdown:** La respuesta de Gemini venía ocasionalmente envuelta en bloques de markdown (````json ... ````) a pesar de la instrucción en el prompt. Se añadió un paso de limpieza en el repositorio que elimina esas marcas con regex antes de parsear el JSON.
- **LaunchedEffect no cancelado:** El efecto typewriter generado por la IA no limpiaba el LaunchedEffect al navegar a otra pantalla, causando que el índice siguiera incrementándose en background. Se añadió la cancelación correcta del coroutine con una key vinculada al texto de la receta.
- **Lista de ingredientes no editable:** La pantalla de ingredientes detectados no permitía editar la lista (era de solo lectura). Se añadió la funcionalidad de eliminar ingredientes con un ícono de cierre en cada chip y un campo de texto para agregar ingredientes adicionales manualmente.
- **Error 6 — OkHttp logging interceptor no resuelve** La dependencia okhttp-logging generaba Unresolved reference 'logging' e Unresolved reference 'HttpLoggingInterceptor' a pesar de estar correctamente declarada en libs.versions.toml. El conflicto era de versiones entre OkHttp y Retrofit 2.11.0. Solución: eliminar completamente el interceptor de logging del proyecto ya que no es funcional para el propósito del taller, simplificando el OkHttpClient a `OkHttpClient.Builder().build()`.
- **Error 7 — Paquete incorrecto en AndroidManifest** Igual que en el Taller 04, el Manifest referenciaba `com.recetas.app` pero el proyecto fue creado con el paquete `com.unilibre.recetasia`. Solución: actualizar namespace y applicationId en `build.gradle.kts` al paquete correcto.
- **Error 8 — API de Gemini no disponible desde Colombia** La API de Google Gemini (`generativelanguage.googleapis.com`) no era accesible desde Colombia, devolviendo error 404 en todas las solicitudes independientemente del modelo o la API Key utilizada. Se intentaron los modelos `gemini-1.5-flash`, `gemini-2.0-flash` y `gemini-1.5-flash-8b` sin éxito. Solución: migrar a una API alternativa.

- **Error 9 — OpenRouter con error 429 en todos los modelos gratuitos** Se migró a OpenRouter como alternativa gratuita, pero todos los modelos gratuitos disponibles (meta-llama/llama-3.2-3b-instruct:free, google/gemma-4-26b-a4b-it:free, baidu/cobuddy:free, entre otros) devolvían error 429 (demasiadas solicitudes) de forma inmediata, incluso en la primera llamada. Esto sugiere que los modelos gratuitos de OpenRouter tienen cuotas muy restrictivas o estaban saturados. Solución: migrar a Groq.
- **Error 10 — Groq error 400 por nombre de modelo incorrecto** Al integrar la API de Groq, el modelo llama3-8b-8192 generaba error HTTP 400. Al verificar en el Playground de Groq se descubrió que el modelo disponible tenía un nombre diferente: llama-3.1-8b-instant. Solución: actualizar el nombre del modelo en GroqRequest.kt.
- **Error 11 — ML Kit reconociendo objetos no relacionados con comida** ML Kit Image Labeling detectaba cualquier objeto visible en la cámara (computadores, mesas, sillas, etc.) y los pasaba como ingredientes a la IA, generando recetas sin sentido. Solución: implementar una lista de filtrado PALABRAS_COMIDA con más de 80 términos relacionados con alimentos en inglés (idioma que usa ML Kit internamente), descartando cualquier etiqueta que no coincida con un ingrediente conocido.

Reflexión Final: IA como Herramienta de Apoyo al Desarrollo

A lo largo de los cinco talleres, la inteligencia artificial demostró ser un colaborador valioso en múltiples frentes del desarrollo Android con Jetpack Compose. Su utilidad fue especialmente notable en la generación de andamiajes iniciales, la sugerencia de estructuras de código y la explicación de conceptos arquitectónicos. Sin embargo, el trabajo con IA también evidenció limitaciones importantes que refuerzan la necesidad del juicio crítico del desarrollador.

En términos de aciertos, la IA fue eficiente para generar componentes visuales a partir de descripciones en lenguaje natural, proponer estructuras de arquitectura correctas (MVVM, Clean Architecture), explicar errores de compilación con diagnósticos precisos y sugerir alternativas cuando una API estaba deprecada. El flujo de trabajo se aceleró significativamente en las fases de scaffolding inicial.

En cuanto a limitaciones, la IA mostró una tendencia recurrente a generar código que compilaba pero no seguía las convenciones actuales (APIs deprecated, versiones incompatibles de librerías), a olvidar detalles especificados en el prompt (como el filtro de confianza de ML Kit), y a producir arquitecturas que mezclaban capas cuando el proyecto crecía en complejidad. Además, el código generado requería siempre una revisión crítica antes de integrarse, ya que ningún resultado fue usado directamente sin modificaciones.

La experiencia confirma el principio pedagógico de estos talleres: la IA es un amplificador de las capacidades del desarrollador, no un sustituto. El valor real está en saber leer, evaluar y corregir el código generado, habilidad que solo se desarrolla con práctica y comprensión profunda de los fundamentos.